

DEVELOPMENT FOUNDATION · V2.0

Product & Platform Specification

The canonical product requirements, platform architecture, evaluation contract, and development baseline for Mendpoint.

Fettler

ReGauge

STATUS

Development foundation
Canonical baseline

UPDATED

14 August 2026
Version 2.0

AUDIENCE

Product · Engineering · Design
FDE · Security · GTM · Diligence

NAVIGATION

Table of Contents

A development contract for how Mendpoint is built, evaluated, operated, and allowed to learn. Major sections begin on a fresh page for durable reference.

0. How to use this document

..... 6

1. Executive summary

.....7

2. Problem definition

..... 8

3. Product vision and north star

..... 9

4. Product boundaries

.....10

5. Users and jobs to be done

..... 12

6. System mental model

..... 13

7. Reference architecture

..... 15

8. Canonical domain model

..... 18

9. Fettler product specification

..... 22

10. ReGauge product specification	
.....	27
11. Mendpoint Change Graph	
.....	31
12. Evidence, confidence, risk, and uncertainty	
.....	34
13. Task orchestration and model router	
.....	36
14. Deterministic recipes and migration playbooks	
.....	38

15. Verification engine	
.....	39
16. Review-first execution and PR delivery	
.....	41
17. Learning and improvement flywheel	
.....	42
18. Synthetic evaluation and pre-design-partner hardening	
.....	46
19. Security, privacy, and governance	
.....	50

20. Reliability and operational behavior

..... 52

21. Performance and scale

..... 53

22. Observability

..... 54

23. User experience specification

..... 56

24. Integrations

..... 58

25. Pricing and Migration Compute Units

..... 59

26. Go-to-market and product sequencing

..... 60

27. Roadmap and release gates

..... 60

28. Acceptance criteria

..... 62

29. Design-partner readiness scorecard

..... 64

30. Risk register

..... 64

31. Development principles

..... 65

32. Architecture decision requirements

.....67

33. Open product decisions

.....67

34. Glossary

..... 68

35. Canonical end-to-end flows

.....70

36. Definition of product maturity

.....72

37. Final product statement

.....73

0. How to use this document

This document is the canonical product and platform specification for future Mendpoint development unless superseded by an approved architecture decision record, product decision, or newer version of this specification.

The previous product specification established the core thesis, the Fettle/ReGauge product split, graph-scoped reasoning, review-first execution, hybrid model orchestration, migration data as a compounding asset, and the land-and-expand product strategy. This revision preserves those decisions and formalizes the missing contracts required to build, evaluate, operate, and evolve the platform coherently.

Where this document introduces detail that was not explicit in the previous draft, it should be treated as a **development baseline**, not as a claim about what is already implemented. Existing production behavior remains the implementation source of truth until intentionally migrated.

0.1 Normative language

The words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, and **MAY** are used deliberately:

- **MUST / MUST NOT** define platform invariants or release-blocking behavior.
- **SHOULD / SHOULD NOT** define the preferred implementation unless evidence justifies an exception.
- **MAY** identifies optional or context-dependent behavior.

0.2 Canonical naming

The canonical product names are:

- **Fettle** — external-change detection, impact analysis, and remediation.
- **ReGauge** — internal modernization, legacy migration, and staged transformation campaigns.

Historical identifiers such as `Warden` or `Transformer` may still exist in code, schemas, migrations, telemetry, or APIs. They **MUST NOT** be blindly renamed where doing so would break persistence or compatibility. New customer-facing product language, new documentation, and new product-level abstractions **MUST** use Fettle and ReGauge.

0.3 Product invariants

These invariants define Mendpoint more strongly than any individual feature:

1. **Migration-specific, not general-purpose.** Mendpoint exists to understand, plan, execute, verify, and govern code change adoption and migration work.
2. **Graph-scoped.** Material recommendations **MUST** be grounded in an explicit representation of affected systems, code, dependencies, and evidence.
3. **Review-first.** Mendpoint **MUST NOT** silently merge or deploy production changes as the default product behavior.
4. **Evidence-backed.** Every material finding, recommendation, and edit **SHOULD** be traceable to source evidence and verification.

5. **Risk-aware.** Higher-blast-radius and lower-confidence work **MUST** receive stronger verification, model escalation, or human review.
6. **Model-neutral.** No product capability may depend conceptually on one model provider remaining best in class.
7. **Reproducible.** Important decisions, graph states, model selections, edits, and verification results **MUST** be replayable or reconstructable to a practical degree.
8. **Governed learning.** Customer and synthetic outcomes **MAY** improve Mendpoint, but learning **MUST** respect provenance, consent, tenant isolation, residency, evaluation holdouts, and promotion controls.
9. **No benchmark gaming.** Evaluation failures **MUST** drive generalized capability fixes rather than scenario-specific exceptions.
10. **Economic discipline.** Quality is the primary constraint, but latency, model cost, verification cost, and frontier-model escalation are first-class product metrics.

1. Executive summary

Mendpoint is an AI-native migration platform that turns software change into safe, reviewable migration work.

It has two tightly connected products:

Fettler

Fettler monitors and interprets changes originating outside the customer's codebase, including API changes, SDK releases, contract changes, deprecations, release notes, and provider migration requirements. It maps those changes to affected customer code, determines blast radius, proposes precise remediation, verifies the remediation, and packages the result as reviewable engineering work.

Its core loop is:

```
external change
→ normalize
→ understand semantic impact
→ map to customer code
→ generate remediation
→ verify
→ explain
→ open reviewable PR
→ learn from outcome
```

ReGauge

ReGauge handles changes originating from the customer's modernization intent: framework upgrades, runtime migrations, SDK rewrites, internal standardization, architecture migrations, and legacy transformation programs. It reconstructs the relevant system, identifies constraints and dependencies, creates a staged plan, executes migration work incrementally, verifies each stage, and tracks exceptions and campaign progress.

Its core loop is:

```
modernization objective
→ reconstruct system
→ identify constraints and dependencies
→ plan stages
→ execute bounded changes
→ verify
→ review
→ advance campaign
→ learn from outcome
```

Shared platform

Fettler and ReGauge share one intelligence and execution platform:

```
integrations
+ change normalization
+ Mendpoint Change Graph
+ evidence model
+ task decomposition
+ model/router layer
+ deterministic tools and recipes
+ code execution and verification
+ review workflow
+ policy/governance
+ evaluation
+ learning flywheel
+ observability
```

The platform thesis is that migration should stop being ad hoc project work and become an operating capability: observable, scoped, explainable, repeatable, and increasingly automated without sacrificing review or trust.

2. Problem definition

Software changes continuously while the systems depending on it do not update themselves.

The failure appears in two directions.

2.1 External change debt

Providers change APIs, SDKs, contracts, authentication requirements, schemas, semantics, and supported versions. Customers often discover the consequences late because:

- release notes are not mapped to actual call sites;
- OpenAPI or SDK diffs do not express business impact by themselves;
- deprecations remain live long after notification;
- behavior changes may compile while still being operationally wrong;
- package upgrade tooling catches version changes but not provider semantics;
- platform teams cannot manually assess every repository every time a provider changes;

- general coding agents require humans to first discover, scope, and explain the migration.

Fettler addresses this problem.

2.2 Internal migration debt

Organizations also accumulate internal migration work:

- runtimes fall behind;
- frameworks become unsupported;
- internal SDKs evolve;
- architecture standards change;
- authentication and security conventions shift;
- business logic becomes entangled with infrastructure;
- monoliths and service ecosystems accumulate undocumented dependencies;
- modernization programs become too large to execute safely as one project.

ReGauge addresses this problem.

2.3 Why existing categories are incomplete

Mendpoint sits between several existing tool categories without reducing to any one of them:

- API management tools govern APIs but generally do not repair downstream customer code.
- Dependency bots update versions but generally do not understand semantic provider changes or organization-specific migration intent.
- Static analysis tools identify patterns but generally do not own migration planning and verified remediation.
- General coding agents can edit code but do not inherently provide migration-specific graph scoping, campaign semantics, evidence, risk controls, outcome learning, or provider change ingestion.
- Consulting and manual migration programs can handle complexity but do not compound software intelligence across repeated migrations.

Mendpoint's category is therefore best described as a **migration operating layer** or **migration platform**.

3. Product vision and north star

3.1 Vision

Mendpoint becomes the system of record and execution layer for code change adoption.

In the target state:

- Fettler is the trusted interface between external platform change and customer code.
- ReGauge is the trusted interface between modernization intent and large-scale internal code evolution.
- Every migration has an explicit scope, graph, evidence trail, plan, review path, verification history, and measurable outcome.

- Organizations can answer what must change, why, where, in what order, with what risk, and whether the change actually succeeded.
- Repeated migrations improve Mendpoint's graph intelligence, recipes, router policy, verification heuristics, and specialized models.
- Frontier models are used where their reasoning quality is justified, not as an undifferentiated default for every token.
- Migration knowledge becomes an institutional asset rather than disappearing into one-off project history.

3.2 North-star outcome

The north-star product outcome is:

A required software migration can move from detected or declared change to verified, reviewable, correctly scoped code updates with materially less human discovery, coordination, and implementation effort than the incumbent process.

3.3 Product success is not code generation

Mendpoint is successful only when generated migration work is:

- correctly scoped;
- technically correct;
- verifiable;
- understandable to reviewers;
- safe relative to its blast radius;
- easy to accept or correct;
- operationally useful;
- cheaper and faster than the manual alternative;
- a source of reliable learning for the next migration.

Generating a patch is an intermediate step, not the end state.

4. Product boundaries

4.1 In scope

Mendpoint is in scope when the work can be represented as a migration or change-adoption problem.

Examples:

- third-party API changes;
- SDK and library migrations;
- API deprecations;
- provider capability adoption;

- runtime upgrades;
- framework upgrades;
- internal SDK migrations;
- organization-wide convention changes;
- authentication migrations;
- service boundary modernization;
- migration campaigns across many repositories;
- architecture transformations that can be decomposed into reviewable stages.

4.2 Explicit non-goals

Mendpoint is not intended to become:

- a full IDE replacement;
- a general-purpose autonomous software engineer;
- a generic issue-to-code agent for arbitrary feature development;
- an automatic production deployment system;
- a system that merges high-risk changes without review;
- a generic enterprise workflow automation platform;
- a proprietary GPU-training infrastructure company;
- a model lab dependent on training a frontier foundation model from scratch.

4.3 Scope test for new features

A proposed feature belongs in Mendpoint if it materially improves one or more of:

1. detecting a required migration;
2. understanding migration scope;
3. reconstructing dependencies or constraints;
4. planning migration sequence;
5. generating migration edits;
6. verifying migration correctness;
7. reviewing or governing migration work;
8. measuring migration progress or outcome;
9. learning reusable migration intelligence.

Features that do not satisfy this test require explicit product justification.

5. Users and jobs to be done

5.1 Primary users

Engineering managers

Need to understand migration scope, staffing burden, risk, progress, and acceptance without manually coordinating every repository.

Staff and senior engineers

Need accurate impact analysis, evidence, safe code changes, and clear exception handling.

Platform and developer productivity teams

Need organization-wide migration visibility and a repeatable way to execute upgrades across many repositories.

Security and compliance teams

Need auditable changes, policy enforcement, approval records, and evidence of verification.

Forward deployed engineers and solution architects

Need to launch and manage high-value provider or enterprise migration campaigns without rebuilding the migration process per customer.

5.2 Secondary users

- API providers seeking faster customer adoption.
- CTOs and VP Engineering teams managing technical risk and modernization debt.
- Product and solution engineering teams coordinating migration programs.
- Reliability teams managing deprecations and operational risk.

5.3 Core jobs

Users hire Mendpoint to answer:

- What changed?
- Does this change affect us?
- Where does it affect us?
- What will break?
- What else depends on the affected code?
- What should be changed?
- Can Mendpoint make the change safely?
- How do we know the change is correct?
- In what order should a large migration happen?
- What remains unresolved?

- What evidence can a reviewer trust?
- What did we learn that should improve the next migration?

6. System mental model

6.1 Mission as the shared execution primitive

A **Mission** is the primary unit of Mendpoint work.

A Mission has:

- a trigger or objective;
- tenant and repository scope;
- source evidence;
- current graph snapshot references;
- one or more migration tasks;
- risk and confidence context;
- execution history;
- verification results;
- review state;
- final outcome;
- learning provenance.

Fettler and ReGauge use the same Mission abstraction but create missions differently.

Fettler mission

Created from an external change event or campaign.

```
ProviderChange  
→ FettlerMission  
→ ImpactFindings  
→ MigrationTasks  
→ CandidateEdits  
→ Verification  
→ Reviewable PR(s)
```

ReGauge mission

Created from an internal migration objective.

```
MigrationObjective
→ ReGaugeMission
→ Discovery
→ MigrationGraph
→ MigrationPlan
→ Stages
→ MigrationTasks
→ PR_batches
→ Verification
```

6.2 Campaign

A **Campaign** coordinates multiple related Missions or stages.

Examples:

- a provider-wide migration across 200 repositories;
- a Python runtime upgrade across 40 services;
- an internal authentication standard migration;
- a multi-stage monolith modernization.

Campaigns MUST support:

- explicit scope;
- stage/dependency ordering;
- status roll-up;
- exceptions;
- pause and resume;
- cancellation;
- retry;
- audit history;
- partial success;
- rollback guidance;
- ownership.

6.3 Evidence

An **Evidence** object is a first-class platform primitive representing why Mendpoint believes something.

Evidence MAY originate from:

- provider specifications;
- release notes;
- changelogs;
- SDK diffs;
- repository code;
- dependency manifests;
- static analysis;

- runtime metadata;
- tests;
- CI results;
- historical migrations;
- human review;
- deterministic tool output.

Material findings **MUST** reference their supporting evidence.

6.4 Verification

A **Verification Result** is an observed result of testing or validating a proposed migration.

Verification is separate from model confidence. A model may be confident and wrong; a low-confidence change may verify successfully. The product **MUST** preserve this distinction.

7. Reference architecture

Mendpoint is organized into eight logical layers. Physical services may combine layers where appropriate; the logical boundaries remain useful for design and testing.

7.1 Layer A — Integration and ingestion

Responsibilities:

- GitHub and GitLab connectivity;
- repository metadata and branch access;
- provider source ingestion;
- OpenAPI/spec retrieval;
- changelog and release-note ingestion;
- SDK release metadata;
- internal migration objective intake;
- CI/test system integration;
- optional runtime metadata ingestion.

Key requirements:

- idempotent ingestion;
- source provenance;
- source versioning;
- tenant scoping;
- least privilege;
- replayable raw input references.

7.2 Layer B — Change normalization

Responsibilities:

- convert external change sources into structured change objects;
- classify changes as additive, breaking, behavioral, deprecation, security, auth, schema, version, or unknown;
- preserve raw source evidence;
- deduplicate equivalent change signals;
- identify confidence and ambiguity.

The normalization layer MUST NOT discard raw source evidence needed for later review.

7.3 Layer C — Mendpoint Change Graph

Responsibilities:

- represent providers, contracts, repositories, code structures, dependencies, runtime relationships, tests, owners, migrations, and verification;
- version graph facts over time;
- support blast-radius and dependency-order queries;
- preserve evidence on graph edges;
- expose uncertainty rather than silently converting incomplete evidence into certainty.

The Change Graph is the primary shared intelligence substrate for Fettler and ReGauge.

7.4 Layer D — Mission orchestration and routing

Responsibilities:

- decompose missions into structured tasks;
- construct context packs;
- choose deterministic tools, recipes, specialized models, open models, or frontier models;
- enforce policy;
- manage retries and fallbacks;
- track task-level cost, latency, confidence, and outcome.

Models receive structured task specifications where practical, not unbounded raw prompts.

7.5 Layer E — Execution and verification

Responsibilities:

- scan and analyze code;
- generate candidate edits;
- apply edits in isolated workspaces;
- generate or select tests;
- run compile, lint, unit, integration, contract, and policy checks as available;
- distinguish pre-existing failures from introduced failures where possible;

- package candidate changes for review.

7.6 Layer F — Review and delivery

Responsibilities:

- evidence-backed explanations;
- semantic diff review;
- risk and confidence presentation;
- approvals and rejections;
- reviewer corrections;
- draft PR creation;
- campaign and exception management;
- retry and rollback controls.

7.7 Layer G — Evaluation and learning

Responsibilities:

- capture outcomes;
- generate learning events;
- classify lessons;
- maintain synthetic and production evaluation datasets;
- manage train/validation/holdout boundaries;
- generate corpora;
- integrate external trainers;
- register candidate adapters/models;
- run held-out evaluation and canaries;
- feed verified outcomes into router policy, graph improvements, recipes, prompts, tooling, or model training.

7.8 Layer H — Platform, security, governance, and observability

Responsibilities:

- tenant isolation;
- authorization;
- encryption;
- secret redaction;
- audit logging;
- residency controls;
- consent and training eligibility;
- trace storage;
- cost accounting;
- health monitoring;
- incident response;

- feature flags;
- release controls.

8. Canonical domain model

The names below are logical contracts. Existing implementation names MAY differ. New implementations SHOULD map to these concepts.

8.1 Tenant

Represents an organization boundary.

Required properties:

```
tenant_id
name
policy_profile
residency_policy
learning_consent_policy
retention_policy
created_at
```

8.2 Integration

Represents an authenticated connection to GitHub, GitLab, CI, a provider source, or another supported system.

Required properties:

```
integration_id
tenant_id
type
scope
auth_reference
permissions
health
created_at
updated_at
```

Credentials MUST NOT be embedded in normal trace payloads.

8.3 ProviderChange

Structured external change consumed by Fettler.

Suggested shape:

```
{
  "change_id": "chg...",
  "provider": "example-provider",
  "source_type": "openapi_diff",
  "source_versions": {"from": "v1", "to": "v2"},
  "change_class": "breaking",
  "affected_objects": [],
  "summary": "",
  "raw_evidence_refs": [],
  "confidence": 0.0,
  "detected_at": ""
}
```

8.4 MigrationObjective

Structured modernization intent consumed by ReGauge.

Suggested shape:

```
{
  "objective_id": "obj...",
  "tenant_id": "t...",
  "title": "",
  "target_state": "",
  "repository_scope": [],
  "constraints": [],
  "success_conditions": [],
  "risk_tolerance": "",
  "requested_by": "",
  "created_at": ""
}
```

8.5 RepositorySnapshot

Represents the repository state against which analysis and edits were produced.

A snapshot **MUST** be immutable by reference and **SHOULD** include:

```
repository_id
branch
commit_sha
captured_at
language/runtime metadata
manifest references
index version
graph version
```

8.6 Mission

Suggested state model:

```
CREATED
→ DISCOVERING
→ SCOPED
→ PLANNING
→ EXECUTING
→ VERIFYING
→ AWAITING_REVIEW
→ ACCEPTED / REJECTED / PARTIAL / FAILED / CANCELLED
```

Fettler MAY skip explicit planning for small changes. ReGauge generally SHOULD retain explicit planning.

8.7 MigrationTask

The structured work unit sent to tools/models.

Required fields SHOULD include:

```
task_id
mission_id
task_type
product
repository_snapshot
graph_context
evidence_refs
constraints
risk_class
verification_requirements
cost/latency_policy
allowed_tools
expected_output_schema
```

8.8 ImpactFinding

Represents an identified affected object or code path.

Required fields:

```
finding_id
change_or_objective_ref
repository
location
affected_entity
impact_type
evidence_refs
graph_path
risk
confidence
status
```

8.9 CandidateEdit

Represents a proposed code change before review.

Required fields:

```
candidate_id
task_id
base_snapshot
patch_ref
changed_files
rationale
evidence_refs
risk
model_or_recipe
verification_state
```

8.10 VerificationResult

Required fields:

```
verification_id
candidate_id
check_type
command_or_test_ref
environment_ref
status
baseline_status
introduced_failure
output_ref
duration
created_at
```

8.11 ReviewDecision

Required fields:

```
review_id
candidate_or_pr_ref
reviewer
decision
comments
substantive_edits
policy_overrides
created_at
```

8.12 LearningEvent

A governed representation of a verified lesson-producing outcome.

Suggested shape:

```
{
  "event_id": "learn...",
  "product": "fettler",
  "mission_id": "",
  "task_type": "",
  "capability": "",
  "model": "",
  "adapter": "",
  "router_decision": {},
  "input_refs": [],
  "prediction": {},
  "observed_outcome": {},
  "verification": {},
  "reviewer_decision": {},
  "correction": {},
  "confidence": null,
  "cost": {},
  "latency": {},
  "provenance": {},
  "consent": {},
  "tenant": {},
  "residency": {},
  "source_class": "SYNTHETIC_GROUND_TRUTH",
  "created_at": ""
}
```

Private chain-of-thought MUST NOT be stored as learning data.

9. Fettler product specification

9.1 Purpose

Fettler detects external software change, determines whether and where it affects connected codebases, generates safe remediation, verifies the remediation, and produces reviewable migration work.

9.2 Fettler triggers

Fettler MUST support an extensible trigger model. Initial trigger classes include:

- OpenAPI diff;
- SDK release or changelog;
- release note;
- deprecation notice;
- provider security/authentication change;
- manual provider announcement;
- customer-reported incident linked to provider behavior;
- scheduled provider migration campaign.

9.3 Change taxonomy

Fettler SHOULD normalize external changes into one or more of:

```
ADDITIVE  
BREAKING_CONTRACT  
BEHAVIORAL  
DEPRECATION  
AUTHENTICATION  
AUTHORIZATION  
SCHEMA  
ENUM  
PAGINATION  
RATE_LIMIT  
WEBHOOK  
SDK_SURFACE  
VERSION_REQUIREMENT  
SECURITY  
END_OF_LIFE  
UNKNOWN
```

Classification MUST preserve ambiguity. Unknown or conflicting evidence SHOULD be represented rather than forced into a known category.

9.4 Fettler workflow

Stage F1 — Ingest

- retrieve or receive source;
- verify source identity where possible;
- store immutable evidence reference;
- deduplicate;
- establish source version boundary.

Stage F2 — Normalize

- identify changed provider objects;
- classify semantic change;
- extract migration guidance if present;
- assign preliminary severity/confidence;
- create ProviderChange.

Stage F3 — Candidate impact discovery

- query provider-to-code graph relationships;
- search repository references;
- inspect SDK usage;
- inspect wrappers and internal abstractions;
- identify dynamic or ambiguous paths requiring deeper analysis.

Stage F4 — Blast-radius analysis

For every candidate finding, determine:

- direct call sites;
- wrapper or abstraction layers;
- dependent services/modules;
- tests touching the path;
- ownership where available;
- runtime or environment relevance;
- likely migration order.

Stage F5 — Task decomposition

Create bounded MigrationTasks for:

- evidence confirmation;
- code transformation;
- test generation;
- verification;
- explanation;
- campaign sequencing.

Stage F6 — Remediation generation

Prefer, in order where appropriate:

1. deterministic transformation recipe;
2. specialized adapter/model;
3. general code model;
4. frontier model escalation.

Generated edits MUST respect forbidden zones, repository policy, and scoped files.

Stage F7 — Verification

Run the strongest available verification appropriate to risk and repository support.

Stage F8 — Review packaging

Produce:

- impact summary;
- changed files;
- semantic change explanation;
- evidence;
- risk;
- confidence;
- verification result;
- unresolved uncertainty;

- rollback guidance if material;
- draft PR.

| Stage F9 — Outcome capture

Record:

- accepted/rejected;
- reviewer corrections;
- merge outcome;
- verification after correction;
- later failure signals where available;
- learning eligibility.

9.5 Fettler functional requirements

| FET-001 — Source provenance

Every ProviderChange MUST retain references to source evidence.

| FET-002 — Semantic classification

Fettler MUST distinguish syntactic version change from semantic migration significance.

| FET-003 — Repository precision

Impact findings MUST identify repository and concrete code location when available.

| FET-004 — Graph path

Material impact findings SHOULD expose the dependency path explaining why the change matters.

| FET-005 — False-positive control

Fettler MUST support explicit dismissal of false impact findings and capture them as learning signals when eligible.

| FET-006 — Hidden usage

Fettler SHOULD detect provider usage through wrappers, generated clients, shared libraries, and common indirection patterns where graph/index support exists.

| FET-007 — Edit generation

Fettler SHOULD generate precise candidate edits rather than only prose recommendations for supported migration classes.

| FET-008 — Verification

Candidate edits MUST expose verification status before review.

| FET-009 — Baseline failures

Where feasible, verification SHOULD distinguish pre-existing failures from failures introduced by the candidate edit.

| FET-010 — Review

A reviewer **MUST** be able to accept, reject, request regeneration, or modify a candidate.

| FET-011 — PR packaging

Supported repositories **SHOULD** receive draft PRs with rationale, evidence, risk, and verification summary.

| FET-012 — Campaign mode

Fettler **MUST** support one provider change affecting many repositories without requiring each repository to be operated as an unrelated workflow.

| FET-013 — Partial coverage

Fettler **MUST** surface graph or repository coverage gaps rather than reporting complete safety when evidence is incomplete.

| FET-014 — Audit

Every material inference, model/tool execution, edit, verification action, and review decision **MUST** be auditable.

9.6 Fettler user experience

Primary screens:

- provider/change event feed;
- change event detail;
- affected repository list;
- impact graph;
- impact finding detail;
- candidate edit/PR review;
- campaign status;
- audit trail.

The UI **SHOULD** make the sequence obvious:

```
what changed
→ where we found impact
→ why we believe it
→ what we changed
→ how we verified it
→ what still needs attention
```

9.7 Fettler failure behavior

Fettler **SHOULD** fail safely.

Examples:

- incomplete graph → lower confidence and explicit coverage warning;
- unavailable CI → PR may be generated but verification marked incomplete;
- ambiguous migration guidance → escalate to stronger reasoning or human review;

- conflicting provider evidence → block high-confidence remediation until resolved;
- repository permission loss → stop affected tasks, preserve campaign state, surface integration error;
- model failure → retry/fallback according to policy, never silently claim success.

10. ReGauge product specification

10.1 Purpose

ReGauge turns internal modernization intent into an evidence-backed, dependency-aware, staged migration program.

It is optimized for migrations that are larger, more ambiguous, and more structurally coupled than the typical Fettle event.

10.2 ReGauge objective classes

Initial objective classes include:

```
FRAMEWORK_UPGRADE  
RUNTIME_UPGRADE  
INTERNAL_SDK_MIGRATION  
EXTERNAL_SDK_MIGRATION  
AUTH_STANDARDIZATION  
CODEBASE_STANDARDIZATION  
INFRASTRUCTURE_MIGRATION  
SERVICE_BOUNDARY_CHANGE  
ARCHITECTURE_TRANSFORMATION  
LEGACY_DECOMPOSITION  
VENDOR_EXIT  
CUSTOM_MIGRATION
```

10.3 ReGauge workflow

Stage R1 — Objective intake

Capture:

- desired target state;
- repository scope;
- business/technical constraints;
- deadlines if material;
- forbidden zones;
- risk tolerance;
- success conditions.

Stage R2 — Discovery

Build or refresh:

- repository topology;
- module/service relationships;
- dependency graph;
- runtime/version inventory;
- test coverage relationships;
- ownership;
- configuration and deployment boundaries;
- architecture conventions;
- known exceptions.

Stage R3 — Constraint analysis

Identify:

- hard blockers;
- incompatible dependencies;
- cycles;
- shared state;
- database coupling;
- undocumented runtime dependencies;
- feature flags;
- sequencing constraints;
- unsupported components;
- ambiguous areas requiring human confirmation.

Stage R4 — Migration graph

Create a graph of migration units and ordering constraints.

A migration unit SHOULD be independently reviewable where possible.

Stage R5 — Plan

Produce:

- target architecture or target state;
- stages;
- dependencies;
- expected PR batches;
- validation strategy;
- rollback/escape plan;
- exceptions requiring manual handling.

Stage R6 — Execute stage

For each stage:

- create bounded tasks;
- generate edits;
- run verification;
- produce PR batch;
- collect review;
- update campaign state.

Stage R7 — Re-plan as evidence changes

ReGauge MUST support plan revision when:

- hidden dependencies emerge;
- verification contradicts assumptions;
- reviewers change constraints;
- a migration stage fails;
- architecture drift is discovered.

Stage R8 — Complete campaign

Produce:

- completed stages;
- unresolved exceptions;
- residual legacy surface;
- verification status;
- migration history;
- executive modernization summary.

10.4 ReGauge functional requirements

REG-001 — Objective structure

Migration intent MUST be converted from free-form request into a structured objective before high-impact execution.

REG-002 — Topology discovery

ReGauge MUST establish a versioned repository/system snapshot before planning.

REG-003 — Constraint visibility

Material assumptions and unresolved constraints MUST be visible to reviewers.

REG-004 — Dependency ordering

Migration stages MUST preserve known dependency constraints.

REG-005 — Incremental execution

ReGauge SHOULD prefer bounded, incremental PRs over monolithic rewrites.

REG-006 — Plan/edit separation

For high-complexity migrations, plan approval SHOULD be separable from code generation.

REG-007 — Organization conventions

ReGauge SHOULD preserve organization-specific code, architecture, and policy conventions where they are available and allowed.

REG-008 — Exceptions

Unsupported or unsafe areas MUST become explicit exceptions, not silently omitted work.

REG-009 — Graph incompleteness

When the graph is incomplete, ReGauge MUST surface uncertainty and SHOULD reduce automation level.

REG-010 — Campaign state

Campaigns MUST support pause, resume, retry, cancellation, partial completion, and exception tracking.

REG-011 — Stage verification

A stage SHOULD NOT advance automatically when required verification has failed.

REG-012 — Rollback guidance

High-risk stages MUST include rollback or recovery guidance before approval.

REG-013 — Drift

ReGauge SHOULD detect when the codebase changes materially relative to the snapshot used for planning and require revalidation.

REG-014 — Executive reporting

Large campaigns SHOULD produce a concise status view for engineering leadership in addition to developer-level detail.

10.5 ReGauge user experience

Primary surfaces:

- migration objective intake;
- discovery status;
- system topology;
- migration blueprint;
- stage/dependency graph;
- PR batch queue;
- exceptions register;
- campaign progress;

- verification history;
- executive modernization report.

The main UX question is:

What is the safest next migration step, and what evidence supports it?

11. Mendpoint Change Graph

11.1 Purpose

The Change Graph is the shared representation Mendpoint uses to connect:

```
change
→ code
→ dependency
→ runtime path
→ test
→ owner
→ migration
→ verification
→ outcome
```

The graph is not only a code graph. It is a migration knowledge graph.

11.2 Core entity classes

The graph SHOULD support at least:

External entities

- provider;
- API;
- endpoint;
- schema;
- field;
- webhook;
- SDK;
- SDK method;
- provider version;
- changelog item;
- deprecation.

Code entities

- organization;

- repository;
- branch/snapshot;
- service;
- package/module;
- file;
- class;
- function/method;
- call site;
- configuration object;
- database/table where discoverable;
- job/worker;
- test;
- CI workflow.

Organizational entities

- team;
- owner;
- environment;
- policy;
- migration campaign.

Migration entities

- change;
- objective;
- mission;
- task;
- finding;
- candidate edit;
- verification result;
- PR;
- review decision;
- learning event;
- recipe;
- model/adapter.

11.3 Edge classes

Examples:


```
CALLS
IMPORTS
DEPENDS_ON
IMPLEMENTS
WRAPS
USES_ENDPOINT
USES_SDK_METHOD
PRODUCES
CONSUMES
READS
WRITES
DEPLOYS_TO
OWNED_BY
TESTED_BY
DEPRECATED_BY
REPLACED_BY
IMPACTED_BY
MIGRATED_BY
VERIFIED_BY
REVIEWED_BY
DERIVED_FROM
BLOCKS
PRECEDES
```

Edges SHOULD carry:

- source;
- timestamp/version;
- confidence;
- evidence reference;
- extraction method;
- tenant.

11.4 Temporal versioning

The graph MUST support reasoning about change over time.

At minimum Mendpoint must be able to distinguish:

- graph state when a mission was created;
- graph state when a candidate edit was generated;
- graph state after repository changes;
- provider contract version before/after a change.

11.5 Evidence-backed edges

An edge SHOULD NOT be treated as equally trustworthy regardless of origin.

Examples:

- direct parsed import → high deterministic confidence;
- model-inferred architectural relation → probabilistic;
- runtime trace → high observed confidence for that execution context;

- reviewer-confirmed relation → human-verified;
- stale repository snapshot → degraded temporal confidence.

11.6 Required query primitives

The graph layer SHOULD expose stable query capabilities for:

| Blast radius

"What depends directly or transitively on X?"

| Provider usage

"Where is provider object X used, including wrappers?"

| Verification coverage

"What tests or checks cover this affected path?"

| Migration order

"What must change before Y?"

| Constraint discovery

"What cycles or shared dependencies block this migration?"

| Ownership

"Who owns affected components?"

| Confidence

"What important graph relationships remain uncertain?"

11.7 Graph completeness

Mendpoint MUST distinguish:

```
no known impact
```

from:

```
complete evidence of no impact
```

Graph incompleteness is a first-class state.

12. Evidence, confidence, risk, and uncertainty

These concepts MUST remain separate.

12.1 Evidence

Evidence answers:

What supports the claim?

12.2 Confidence

Confidence answers:

How strongly does Mendpoint believe the claim given available evidence?

Confidence SHOULD be calibrated by task family and model/tool history rather than treated as an arbitrary language-model self-score.

12.3 Risk

Risk answers:

What is the consequence if this action or inference is wrong?

Risk SHOULD consider:

- blast radius;
- production criticality;
- security relevance;
- data sensitivity;
- rollback difficulty;
- number of repositories/services;
- runtime coupling;
- verification strength.

12.4 Coverage

Coverage answers:

How much of the relevant system did Mendpoint actually inspect or understand?

A high-confidence result with low coverage MUST NOT be presented as equivalent to a fully scoped result.

12.5 Suggested confidence object

```
{
  "score": 0.86,
  "basis": [
    "deterministic_reference",
    "verified_graph_path",
    "passing_contract_test"
  ],
  "coverage": 0.72,
  "calibration_bucket": "task_family_x",
  "known_unknowns": []
}
```

The exact implementation may differ, but the semantic separation SHOULD remain.

13. Task orchestration and model router

13.1 Principle

Fettler and ReGauge are orchestrators over a shared migration intelligence stack. They SHOULD NOT directly hard-code one model for an entire mission.

13.2 Structured TaskSpec

The router SHOULD receive a TaskSpec containing:

```
product
task_type
capability
repository context
provider/framework context
risk class
blast radius
graph confidence
required tools
verification requirements
latency target
cost ceiling
candidate execution paths
```

13.3 Execution paths

The router may choose:

1. deterministic rule;
2. migration recipe;
3. static/dynamic analysis tool;

4. specialized open model;
5. post-trained vendor/framework/general migration adapter;
6. stronger general model;
7. frontier model.

A task MAY use multiple paths sequentially.

13.4 Router objective

The router's optimization target is:

Select the lowest-cost execution path that reliably meets the task's risk-adjusted quality requirement, with escalation when evidence or confidence is insufficient.

Quality gates override cost savings.

13.5 Router inputs

The router SHOULD learn from:

- task family;
- provider/framework;
- language/runtime;
- repository size;
- context size;
- graph complexity;
- risk;
- blast radius;
- historical model accuracy;
- historical verification success;
- reviewer acceptance;
- latency;
- token cost;
- inference cost;
- escalation history.

13.6 Router outputs

Every routing decision SHOULD produce:

```
selected path
selected model/adaptor/tool
fallback chain
policy rationale
expected quality threshold
cost/latency estimate where available
```

The rationale is a structured product decision record, not private chain-of-thought.

13.7 Escalation

Escalation SHOULD occur when:

- confidence falls below policy threshold;
- graph coverage is incomplete for high-risk work;
- verification repeatedly fails;
- the task is novel relative to known adapter coverage;
- repository context exceeds specialized model capability;
- risk policy requires frontier-level reasoning or human review.

13.8 Model neutrality

The product specification MUST NOT depend on specific model vendors or model names. Model choices are operational configuration governed by evaluation evidence, licensing, cost, latency, and capability.

14. Deterministic recipes and migration playbooks

14.1 Purpose

Repeated migration patterns SHOULD become deterministic recipes when doing so is safer, cheaper, and more explainable than generative execution.

Examples:

- known API field rename;
- canonical SDK method replacement;
- framework configuration migration;
- mechanical import rewrite;
- organization-standard auth middleware transformation.

14.2 Recipe requirements

A production recipe SHOULD include:

```
recipe_id
version
applicability predicate
required evidence
preconditions
transformation
verification requirements
rollback behavior
known limitations
source/provenance
approval status
```

14.3 Recipe selection

Recipes MUST NOT be applied merely because code text resembles a known pattern. Applicability SHOULD be established using graph/context evidence.

14.4 Learning recipes

Repeated verified migration outcomes MAY produce recipe candidates. Candidates require evaluation before normal router eligibility.

15. Verification engine

15.1 Purpose

Verification converts "the edit looks plausible" into observed evidence.

15.2 Verification classes

Mendpoint SHOULD support:

- syntax/parse;
- compile/build;
- lint/static checks;
- unit tests;
- integration tests;
- contract tests;
- provider-specific validation;
- schema validation;
- security/policy checks;
- smoke tests;
- generated tests;
- migration-specific assertions.

15.3 Baseline comparison

Where feasible, Mendpoint SHOULD run verification against both:

- the base repository snapshot;
- the candidate edit.

This enables classification of:

```
PRE_EXISTING_FAILURE
INTRODUCED_FAILURE
RESOLVED_FAILURE
NEW_PASS
UNKNOWN
```

15.4 Risk-adaptive verification

Higher-risk tasks SHOULD require stronger verification. Verification policy SHOULD be configurable by:

- tenant;
- repository;
- risk class;
- migration type;
- provider/framework;
- campaign.

15.5 Verification gaps

Passing tests MUST NOT be presented as proof of semantic safety if meaningful verification coverage is absent.

The UI SHOULD show both:

- what passed;
- what could not be verified.

15.6 Isolated execution

Code execution SHOULD occur in isolated environments with:

- bounded permissions;
- secrets controls;
- network policy;
- resource limits;
- timeouts;
- captured logs;
- reproducible environment metadata where practical.

16. Review-first execution and PR delivery

16.1 Review invariant

The default execution boundary is a reviewable artifact, not production.

16.2 Review package

Every material candidate SHOULD expose:

- what triggered the change;
- affected code;
- graph path;
- rationale;
- source evidence;
- patch;
- risk;
- confidence;
- coverage;
- verification;
- unresolved uncertainty;
- model/tool/recipe provenance;
- rollback note when relevant.

16.3 Semantic review

Review UI SHOULD organize changes by migration meaning where possible, not only file order.

Example categories:

```
contract update
call-site adaptation
configuration change
test update
compatibility shim
cleanup
```

16.4 Review actions

Supported actions SHOULD include:

```
approve
reject
edit
request regeneration
request stronger verification
request escalation
mark false positive
defer
```

Substantive reviewer edits SHOULD be retained as learning signals when eligible.

16.5 PR behavior

Mendpoint SHOULD:

- create a dedicated branch;
- preserve base commit identity;
- use deterministic, descriptive commit/PR metadata;
- include evidence and verification in PR body;
- link the PR to mission/campaign;
- update mission state from PR outcomes where integration permits.

17. Learning and improvement flywheel

17.1 Objective

Every verified migration outcome should be able to improve the next migration without turning production into an uncontrolled self-modifying system.

The learning flywheel is broader than model fine-tuning.

A lesson may improve:

```
model weights
router policy
retrieval
Change Graph construction
parser/static analysis
tool selection
deterministic recipe
prompt/system policy
confidence calibration
product logic
```

17.2 LearningEvent

Learning events MUST preserve:

- provenance;

- product;
- task;
- model/tool/recipe;
- router decision;
- observable output;
- verification;
- reviewer action;
- correction;
- cost/latency;
- tenant/consent/residency eligibility.

17.3 Provenance classes

At minimum the platform SHOULD distinguish:

```
SYNTHETIC_GROUND_TRUTH  
DESIGN_PARTNER_VERIFIED  
PRODUCTION_VERIFIED  
HUMAN_CORRECTED  
DETERMINISTICALLY_VERIFIED
```

These MUST NOT be silently treated as equivalent evidence classes.

17.4 Lesson destination classification

Validated lessons SHOULD be classified into one or more destinations:

```
MODEL_WEIGHT  
ROUTER_POLICY  
RETRIEVAL  
GRAPH  
PARSER  
TOOLING  
DETERMINISTIC_RECIPE  
PROMPT  
PRODUCT_LOGIC  
CALIBRATION  
NO_ACTION
```

Examples:

- missing import due to parser defect → `PARSER`;
- model never saw the required file → `RETRIEVAL`;
- cheap model fails task family while stronger model succeeds → `ROUTER_POLICY`;
- provider-specific remediation pattern repeatedly verifies → `MODEL_WEIGHT` and/or `DETERMINISTIC_RECIPE`.

Mendpoint SHOULD NOT fine-tune models to compensate for deterministic engineering defects.

17.5 Dataset lifecycle

Eligible model-learning events SHOULD move through a governed lifecycle:

```
OPEN
→ COLLECTING
→ VALIDATING
→ SEALED
→ EXPORTED
→ TRAINING
→ ARCHIVED
```

Equivalent existing states are acceptable.

Sealed datasets MUST be immutable by identity.

17.6 Dataset splits

Every training program MUST preserve:

```
TRAIN
VALIDATION
HOLDOUT
```

Holdout examples MUST NOT contribute to training.

Synthetic evaluation should further distinguish:

```
synthetic-development
synthetic-regression
synthetic-hidden-holdout
```

17.7 Corpus families

Training data SHOULD be partitionable by capability:

```
api_change_classification
impact_analysis
dependency_reasoning
root_cause_analysis
blast_radius
remediation_generation
migration_planning
migration_sequencing
verification_reasoning
tool_selection
confidence_calibration
abstention
router_policy
```

17.8 Adapter strategy

The target architecture supports:

| Vendor adapters

Provider-specific migration patterns, primarily useful to Fettle.

| Framework/runtime adapters

Framework, runtime, or migration-family specialization, primarily useful to ReGauge.

| General migration adapter

Cross-provider/framework migration primitives.

Adapter boundaries should be evidence-driven rather than created merely because a category exists.

17.9 Trainer abstraction

Mendpoint SHOULD remain trainer-neutral.

Logical trainer capabilities:

```
submit
status
reconcile
fetch_artifacts
cancel
```

Mendpoint's strategic asset is migration intelligence, not proprietary training infrastructure.

17.10 Candidate lifecycle

Candidate models/adapters SHOULD move through a governed lifecycle:

```
TRAINING
→ TRAINED
→ EVALUATING
→ EVALUATED
→ SHADOW_READY
→ SHADOW
→ CANARY_READY
→ CANARY
→ AWAITING_APPROVAL
→ APPROVED
→ ROUTER_ELIGIBLE
→ ACTIVE
```

Failure/terminal states SHOULD include:

```
REJECTED
QUARANTINED
ROLLED_BACK
RETIRED
```

Production promotion **MUST** remain governed.

17.11 Promotion requirements

A candidate **MUST NOT** become normally router-eligible unless:

- required held-out evaluations pass;
- no unacceptable high-severity regression exists;
- artifact integrity is valid;
- consent/provenance requirements are valid;
- canary requirements are met where applicable;
- explicit promotion approval is recorded.

17.12 Router learning

Routing outcomes are themselves learning data.

Mendpoint **SHOULD** measure by task family:

```
selected model/tool
success
verification
review acceptance
cost
latency
fallback
escalation
```

The router should improve from this evidence.

18. Synthetic evaluation and pre-design-partner hardening

18.1 Purpose

Before relying on customer repositories to reveal product gaps, Mendpoint **SHOULD** systematically create adversarial synthetic repositories and migrations designed to make Fettler and ReGauge fail.

The objective is not benchmark optics. It is early discovery of real capability gaps.

18.2 Synthetic repository corpus

The evaluation system SHOULD include:

- modern applications;
- monoliths;
- modular monoliths;
- monorepos;
- microservices;
- event-driven systems;
- integration-heavy repositories;
- legacy Java/.NET/Python/PHP patterns;
- mixed-generation architectures;
- AI-generated code failure patterns;
- database-coupled systems;
- dynamic/configuration-driven behavior.

18.3 Ground truth

Every synthetic scenario SHOULD have hidden machine-readable ground truth describing:

- intended fault/change;
- architecture truth;
- dependency truth;
- expected findings;
- acceptable variants;
- false-positive traps;
- blast-radius truth;
- preferred remediation;
- scenario tags and difficulty.

The evaluated product MUST NOT have access to the answer key.

18.4 Mutation engine

The system SHOULD programmatically generate controlled variants such as:

- API field removal/rename;
- enum semantic change;
- authentication changes;
- webhook changes;
- SDK breakage;
- dependency conflicts;
- runtime upgrades;
- race conditions;

- missing timeouts;
- hidden coupling;
- undocumented dependencies;
- dead-looking but live code;
- database-side business logic;
- insecure migrations.

18.5 Difficulty

Scenarios SHOULD span:

```
L1 obvious
L2 realistic
L3 cross-cutting
L4 ambiguous
L5 adversarial
```

The corpus SHOULD NOT be dominated by easy tests.

18.6 Evaluation dimensions

Fettler

Measure:

- change classification;
- impact recall;
- impact precision;
- root-cause correctness;
- blast-radius correctness;
- evidence quality;
- remediation correctness;
- verification quality;
- false positives;
- false negatives;
- confidence calibration;
- latency/cost.

ReGauge

Measure:

- architecture reconstruction;
- dependency discovery;
- hidden-dependency recall;
- migration ordering;
- constraint discovery;

- blast-radius prediction;
- modernization plan quality;
- behavior preservation;
- verification;
- confidence calibration;
- latency/cost.

18.7 Failure taxonomy

Evaluation failures SHOULD map into a reusable taxonomy such as:

```
PARSING_FAILURE
LANGUAGE_SUPPORT_FAILURE
REPOSITORY_MAPPING_FAILURE
ARCHITECTURE_INFERENCE_FAILURE
GRAPH_CONSTRUCTION_FAILURE
DEPENDENCY_DISCOVERY_FAILURE
RETRIEVAL_FAILURE
CONTEXT_SELECTION_FAILURE
PROMPT_FAILURE
REASONING_FAILURE
TOOL_SELECTION_FAILURE
TOOL_EXECUTION_FAILURE
MODEL_ROUTING_FAILURE
MODEL_CAPABILITY_FAILURE
CONFIDENCE_CALIBRATION_FAILURE
ROOT_CAUSE_FAILURE
BLAST_RADIUS_FAILURE
REMEDIATION_FAILURE
PATCH_FAILURE
TEST_GENERATION_FAILURE
FALSE_POSITIVE
FALSE_NEGATIVE
PERFORMANCE_FAILURE
COST_FAILURE
UX_PRESENTATION_FAILURE
```

18.8 Fix loop

Every significant failure SHOULD follow:

```
reproduce
→ diagnose root cause
→ choose generalized fix
→ implement
→ rerun failing scenario
→ run related regressions
→ run full relevant suite
→ validate hidden holdout
→ capture lesson
```

Scenario-specific hacks are prohibited.

18.9 Holdouts

Evaluation **MUST** distinguish:

- development scenarios visible during debugging;
- regression scenarios added after failures;
- hidden holdouts never used for direct fixing.

Product readiness is measured by generalization to holdouts, not only the development set.

18.10 Design-partner readiness gate

Before a new capability is pitched as reliable, the team **SHOULD** be able to answer:

- which repository/migration classes are well supported;
- which are partially supported;
- which are explicitly unsupported;
- what the largest unresolved P0/P1 risks are;
- whether holdout performance supports the claim;
- whether rollback and review controls work;
- what verification coverage is available.

19. Security, privacy, and governance

19.1 Tenant isolation

Tenant boundaries **MUST** be enforced across:

- repositories;
- graph data;
- traces;
- prompts/context;
- datasets;
- training corpora;
- model artifacts where private;
- analytics;
- exports.

A tenant-isolation failure is a release-blocking defect.

19.2 Least privilege

Repository integrations **SHOULD** request only permissions required for configured capabilities.

Write permissions **SHOULD** be separable from read-only analysis when supported.

19.3 Secret handling

Mendpoint **MUST** minimize secret exposure to models and logs.

Controls **SHOULD** include:

- secret scanning/redaction;
- bounded context assembly;
- credential reference indirection;
- encrypted storage;
- log filtering.

19.4 Data residency

Residency policy **MUST** be enforced before data crosses storage, model, trainer, or analytics boundaries.

19.5 Learning consent

Training eligibility **MUST** be separate from operational processing permission.

A customer may permit Mendpoint to process code to deliver the product while withholding permission for shared model training.

19.6 Cross-account learning

Cross-account learning **MUST** only use data explicitly eligible for that purpose.

Private tenant patterns **MAY** be represented through tenant-local retrieval, private recipes, or private adapters where supported.

19.7 Deletion and revocation

The system **MUST** maintain enough lineage to identify which datasets and artifacts used a given learning event where policy requires future exclusion or remediation.

The exact legal retention behavior is policy/configuration dependent and should not be hard-coded into this specification.

19.8 Auditability

Security- and migration-critical actions **MUST** be auditable, including:

- source ingestion;
- repository access;
- graph updates;
- model/tool execution;
- edits;
- verification;
- approvals;

- overrides;
- training dataset inclusion;
- candidate promotion;
- rollback.

20. Reliability and operational behavior

20.1 Idempotency

The following operations SHOULD be idempotent by identity where feasible:

- change ingestion;
- repository indexing;
- mission creation from the same trigger;
- task dispatch;
- PR creation;
- training dispatch;
- adapter registration callbacks.

20.2 Durable state

Mission and campaign state MUST survive worker/process failure.

20.3 Retry policy

Retries SHOULD distinguish:

- transient infrastructure failure;
- deterministic product failure;
- provider/model rate limit;
- invalid input;
- policy rejection.

Deterministic failures SHOULD NOT retry indefinitely.

20.4 Reconciliation

Long-running external jobs such as CI, training, or remote repository operations SHOULD support reconciliation after lost callbacks or worker crashes.

20.5 Concurrency

The platform SHOULD protect against:

- duplicate task execution;

- conflicting PR creation;
- stale snapshot edits;
- concurrent campaign stage advancement;
- duplicate training.

20.6 Drift detection

If the target repository changes materially after analysis, Mendpoint SHOULD detect stale context and revalidate rather than applying old assumptions.

20.7 Graceful degradation

When a subsystem is unavailable:

- graph unavailable → do not claim scoped impact;
- model unavailable → use configured fallback or stop safely;
- CI unavailable → mark verification incomplete;
- provider source unavailable → preserve last known state and surface staleness;
- trainer unavailable → learning pipeline queues or fails without affecting core migration delivery.

21. Performance and scale

The previous specification establishes that small-repository Fettler analysis should complete in minutes rather than hours and that large campaigns should provide useful summary output before deep scans finish.

This revision formalizes the performance model without inventing hard thresholds that have not yet been product-approved.

21.1 Performance budgets

Every task family SHOULD have configurable budgets for:

```
time to first useful result
time to complete
model latency
graph query latency
verification latency
token consumption
inference cost
```

21.2 Progressive results

Large scans and campaigns SHOULD stream useful partial results while preserving status semantics such as:

```
partial
coverage percentage
remaining scope
known unknowns
```

21.3 Repository scale

The evaluation program **MUST** measure degradation across:

- small repositories;
- medium repositories;
- large repositories;
- monorepos;
- multi-repository organizations.

Scale-related failures are product gaps, not merely infrastructure issues.

22. Observability

22.1 Trace model

A mission trace **SHOULD** make it possible to reconstruct:

```
trigger
→ graph/context
→ task decomposition
→ router decision
→ tools/models
→ candidate
→ verification
→ review
→ outcome
```

22.2 Product metrics

Fettler

- time from provider change to first useful impact;
- time to first PR;
- impact precision/recall where ground truth exists;
- PR acceptance;
- merge rate;
- reviewer correction rate;
- regression rate;
- incidents avoided/remediated where measurable.

ReGauge

- time to migration blueprint;
- migration stage completion;
- PR batch acceptance;
- exception rate;
- re-plan rate;
- campaign duration;
- residual legacy surface;
- behavior-preservation failures.

22.3 Model/router metrics

Track:

- selected model/adaptor/tool;
- task family;
- quality/evaluation result;
- verification result;
- reviewer outcome;
- latency;
- tokens;
- cost;
- fallback;
- escalation;
- confidence calibration.

22.4 Graph metrics

Track:

- index freshness;
- graph coverage;
- extraction failures;
- ambiguous edges;
- hidden-dependency discoveries;
- stale graph usage;
- query latency.

22.5 Learning metrics

Track:

- learning events by provenance;
- eligible vs blocked events;

- destination classification;
- dataset volume and diversity;
- sealed datasets;
- training jobs;
- candidate pass/fail;
- holdout improvement;
- rollback;
- adapter usage.

22.6 Business metrics

Retain the existing business metrics:

- MCU consumption per customer;
- gross margin per migration class;
- expansion from Fettler to ReGauge;
- connected repositories;
- logo retention;
- net revenue retention.

23. User experience specification

23.1 Global principles

The product SHOULD:

- show evidence before asking users to trust automation;
- present affected code before proposed edits;
- distinguish confidence, risk, and verification;
- surface uncertainty instead of hiding it;
- organize migration work semantically;
- make the next action obvious;
- support progressive disclosure for deep technical detail;
- allow review without requiring users to understand model internals.

23.2 Main surfaces

Shared

- organization/admin;
- integration management;
- repository inventory;
- audit;

- analytics;
- policy;
- model/automation settings where appropriate.

Fettler

- change feed;
- provider change detail;
- impact view;
- candidate remediation;
- PR review;
- provider campaign dashboard.

ReGauge

- modernization objective intake;
- system discovery;
- migration blueprint;
- dependency/stage map;
- PR campaign;
- exceptions;
- modernization report.

23.3 Evidence UI

Every finding SHOULD answer:

```
Claim
Evidence
Graph path
Confidence
Coverage
Risk
Recommended action
Verification
```

23.4 Confidence UI

Avoid presenting confidence as an unexplained percentage.

Where possible show:

- confidence level;
- evidence basis;
- coverage;
- reasons for escalation or uncertainty.

24. Integrations

24.1 Repository providers

Required initial capabilities

- GitHub repository connection;
- organization/repository scoping;
- branch/commit checkout;
- draft PR creation;
- PR status/readback;
- CI status readback.

Planned/extended

- GitLab equivalent capability.

24.2 Provider change sources

Support an extensible adapter interface for:

- OpenAPI;
- structured changelogs;
- release notes;
- SDK metadata;
- manual source upload;
- provider-specific feeds.

24.3 CI and verification

Integrations SHOULD support both:

- native Mendpoint sandbox verification;
- customer CI invocation/readback.

24.4 Runtime/observability data

Runtime metadata is optional but can materially improve hidden-dependency detection. It MUST remain subject to tenant security and scope controls.

25. Pricing and Migration Compute Units

The existing specification defines a consumption-oriented pricing model centered on a **Migration Compute Unit (MCU)**.

25.1 MCU purpose

MCUs provide a normalized unit for migration work that may include:

- graph analysis;
- retrieval/context construction;
- model inference;
- deterministic transformation;
- verification.

The commercial formula is separate from internal raw cost accounting.

25.2 Requirements

MCU accounting SHOULD be:

- deterministic enough to explain;
- observable per mission/campaign;
- independent of one model vendor's token pricing;
- compatible with both Fettler and ReGauge;
- capable of reflecting more expensive high-risk verification.

25.3 Additional commercial objects

The platform MAY support:

- campaign fees;
- firm-fixed-price migration engagements;
- enterprise minimums;
- provider-sponsored campaigns.

Pricing policy is business configuration, not a hard-coded product invariant.

26. Go-to-market and product sequencing

26.1 Land with Fettle

Fettle remains the preferred entry wedge because external changes are:

- time-sensitive;
- concrete;
- easier to scope;
- easier to demonstrate;
- naturally connected to measurable PR outcomes.

26.2 Expand with ReGauge

After Mendpoint earns trust on bounded migration work, ReGauge expands into:

- runtime upgrades;
- framework migrations;
- SDK migrations;
- internal standards;
- architecture modernization.

26.3 Initial vertical focus

The previous draft recommends a single-vertical-first launch, with payments/fintech APIs as the strongest default initial provider category, while preserving the option to expand cross-API after evidence accumulates.

For development, this should translate into:

- architecture that is provider-agnostic;
- initial evaluation depth on a small set of providers/migration classes;
- no core-system assumptions that payments is the only vertical;
- explicit measurement of whether the initial vertical produces stronger design-partner pull.

27. Roadmap and release gates

Roadmap phases are product gates, not merely dates.

Phase 0 — Internal hardening foundation

Goal: prove the system can be evaluated and improved before customer exposure.

Required:

- shared Mission semantics;
- reliable repository snapshots;
- baseline Change Graph;
- Fettler end-to-end path;
- ReGauge planning path;
- synthetic repo/evaluation harness;
- regression suite;
- learning-event capture;
- auditability.

Exit gate:

- no unresolved known P0 platform defect;
- repeatable internal E2E;
- hidden holdout evaluation exists;
- known support boundaries documented.

Phase 1 — Fettler design-partner wedge

Goal: external API changes become useful, reviewable PRs.

Required:

- GitHub;
- OpenAPI/spec diff ingestion;
- impact analysis;
- selected migration classes;
- candidate PR;
- evidence;
- verification;
- review;
- outcome capture.

Exit gate:

- design-partner-ready support matrix;
- rollback and failure behavior proven;
- meaningful adversarial holdouts;
- product can state what it will and will not handle.

Phase 2 — Fettler campaign scale

Goal: provider changes can be managed across multiple repositories.

Required:

- campaign orchestration;

- multi-provider adapter model;
- better confidence/coverage;
- GitLab if prioritized;
- performance at organization scale;
- richer analytics.

Phase 3 — ReGauge execution

Goal: internal migration objective becomes staged, validated PR campaign.

Required:

- discovery/topology;
- migration graph;
- staged plan;
- selected migration class execution;
- exception handling;
- stage verification;
- campaign dashboard.

Phase 4 — Compounding intelligence

Goal: verified outcomes improve cost and quality.

Required:

- governed datasets;
- trainer integration;
- post-trained candidates;
- holdout evaluation;
- router admission;
- shadow/canary;
- rollback;
- recipe extraction;
- router learning.

28. Acceptance criteria

28.1 Shared acceptance criteria

A production-grade mission MUST:

- be tenant scoped;
- reference an immutable repository snapshot;

- retain evidence;
- expose graph/context provenance;
- have risk and verification policy;
- produce an auditable trace;
- stop safely on policy failure;
- preserve review state;
- capture outcome.

28.2 Fettler acceptance criteria

A supported Fettler migration class is design-partner ready when:

- a provider change can be ingested and semantically classified;
- impacted code is identified with useful precision;
- graph/coverage uncertainty is explicit;
- a reviewable candidate edit can be generated;
- verification status is visible;
- PR packaging works;
- reviewers can accept/reject/edit;
- false positives can be recorded;
- audit history persists;
- hidden holdout results support the claimed capability;
- failure and rollback paths are tested.

28.3 ReGauge acceptance criteria

A supported ReGauge migration class is design-partner ready when:

- a modernization objective becomes a structured objective;
- topology and constraints can be reconstructed to a declared coverage level;
- a dependency-aware staged plan is produced;
- at least one bounded stage can generate a validated PR batch;
- exceptions are explicit;
- stage advancement honors verification policy;
- repository drift can trigger revalidation;
- reviewers can change plan or execution;
- campaign progress is durable;
- hidden holdout results support the claimed capability.

28.4 Learning-system acceptance criteria

The learning system is production ready when:

- Fettler and ReGauge emit canonical learning events;
- provenance classes are preserved;

- tenant/consent/residency rules are enforced;
- lessons can be routed to the correct improvement destination;
- datasets are versioned and sealable;
- holdout leakage is prevented;
- corpora are deterministic by dataset version;
- external training can be safely dispatched/reconciled;
- candidate lineage is complete;
- candidate evaluation compares against baseline;
- bad candidates are rejected;
- approved candidates can be canaried;
- rollback works;
- production outcomes feed the next learning cycle.

29. Design-partner readiness scorecard

Every proposed design-partner capability SHOULD have a one-page scorecard containing:

Capability

Supported languages/stacks

Supported repository patterns

Supported providers/frameworks

Known unsupported patterns

Evaluation scenario count

Hidden holdout status

Impact precision/recall where measurable

Patch verification rate

False-positive rate

Known P0/P1 issues

Latency range

Cost range

Required human review

Rollback behavior

Security limitations

Owner

Last validated commit/version

A capability MUST NOT be represented as broadly supported if the scorecard shows only narrow benchmark coverage.

30. Risk register

Risk	Failure mode	Required mitigation
Incomplete graph	Hidden usage missed	

Risk	Failure mode	Required mitigation
		Evidence-backed coverage, hybrid static/runtime data, uncertainty state, human escalation
False confidence	Plausible but wrong migration	Calibration, verification, risk policy, holdouts
Low trust	Engineers refuse AI-generated PRs	Review-first UX, source evidence, small PRs, visible verification
Frontier cost	Poor gross margin	Router, recipes, specialized models, cost telemetry
Over-broad scope	Product drifts into generic coding	Migration scope test and product invariants
Verification gap	Tests pass but behavior is wrong	Contract/policy checks, coverage display, staged rollout
Benchmark overfit	Internal metrics look strong but customers fail	Hidden holdouts, mutation generation, design-partner verification
Tenant leakage	Cross-customer data exposure	Isolation, negative tests, lineage, least privilege
Training leakage	Holdout enters training	Immutable split identity and dataset lineage
Model regression	New adapter harms production	Baseline comparison, shadow/canary, approval, rollback
Stale analysis	Repository changes after planning	Snapshot pinning and drift detection
Provider ambiguity	Release notes insufficient	Source triangulation, lower confidence, escalation
Migration blast radius	Large PR destabilizes system	Staging, dependency ordering, bounded PRs
Legacy dynamic behavior	Static graph misses runtime dependency	Runtime evidence where available, explicit coverage gaps
Concurrent agents/dev changes	Automation overwrites active work	Worktrees/branches, deliberate merge, versioned evaluation

31. Development principles

31.1 Fix root causes

When evaluation exposes failure, first determine whether the cause is:

```
parser
graph
retrieval
context
tool
router
model
prompt
verification
product logic
UX
```

Do not automatically blame the model.

31.2 Prefer generalized capability

A fix is successful when it improves unseen related scenarios, not only the one that exposed the bug.

31.3 Prefer deterministic mechanisms when possible

If a deterministic parser, graph query, or recipe can solve a problem more reliably than a model, prefer it.

31.4 Preserve provenance

Every output worth trusting or learning from should retain enough provenance to understand how it was produced.

31.5 Make uncertainty inspectable

Do not hide coverage gaps behind prose confidence.

31.6 Keep long-running work resumable

Campaigns, repository analysis, training jobs, and external verification should be durable and reconcilable.

31.7 Avoid duplicate platforms

There should be one canonical implementation for:

- router;
- graph;
- learning-event schema;
- dataset lifecycle;
- evaluation framework;
- audit/trace model.

New modules should extend these rather than create parallel substitutes.

32. Architecture decision requirements

Changes to the following SHOULD require an ADR or equivalent explicit decision:

- canonical Mission state machine;
- Change Graph schema primitives;
- tenant isolation model;
- evidence/confidence semantics;
- router policy architecture;
- repository sandbox model;
- learning provenance classes;
- dataset split/holdout guarantees;
- model artifact promotion lifecycle;
- customer-code training policy;
- persistent identifier renames;
- automatic merge/deployment policy.

An ADR SHOULD include:

```
context
decision
alternatives considered
security impact
data/compatibility impact
migration plan
rollback
evaluation plan
```

33. Open product decisions

These questions are intentionally left open because the prior specification does not resolve them and they should be decided from implementation/design-partner evidence rather than silently assumed.

33.1 Initial provider coverage

Which providers and exact change classes are committed for the first Fettler design partners?

33.2 First ReGauge migration class

Which modernization class will be the first fully supported end-to-end ReGauge execution path?

33.3 Runtime metadata

How much runtime/observability data is required for initial graph completeness versus optional enhancement?

33.4 Enterprise deployment

At what stage are VPC/self-hosted deployment modes required versus roadmap?

33.5 Numeric quality gates

What exact precision, recall, verification, latency, and cost thresholds define design-partner and GA readiness by capability?

These SHOULD be configuration/versioned acceptance gates rather than scattered hard-coded constants.

33.6 Cross-account learning policy

What customer consent and contractual model will govern use of verified production outcomes for shared post-training?

33.7 Private adaptation

When should organization-specific behavior live in retrieval/recipes versus private adapters?

33.8 Provider-side product

When should provider-sponsored migration campaigns become a first-class commercial product rather than an FDE workflow?

34. Glossary

Adapter — A specialized post-trained model artifact or equivalent specialization that can be selected by the router.

Blast radius — The set and severity of systems, code paths, repositories, or runtime behavior that may be affected by a change.

Campaign — A coordinated set of related missions or migration stages.

Candidate Edit — A proposed code change that has not yet been accepted.

Change Graph — Mendpoint's versioned graph connecting provider changes, code, dependencies, tests, owners, migrations, and outcomes.

Coverage — The degree to which Mendpoint has inspected and represented the relevant system scope.

Evidence — A source or observation supporting a finding, recommendation, graph edge, or verification result.

Fettler — Mendpoint's external-change remediation product.

Holdout — Evaluation data excluded from model or product-specific training/debugging and reserved for generalization measurement.

Learning Event — Governed structured outcome data that may create an improvement lesson.

MCU — Migration Compute Unit, Mendpoint's normalized consumption unit for migration work.

Migration Objective — Structured statement of desired internal modernization used by ReGauge.

Migration Recipe — Versioned deterministic transformation playbook for a known migration pattern.

Migration Task — Bounded structured work unit executed by tools, recipes, or models.

Mission — Primary execution unit representing a migration problem and its lifecycle.

ProviderChange — Structured representation of an external API/SDK/contract change.

ReGauge — Mendpoint's internal modernization and legacy migration product.

Risk — Expected consequence if an inference or action is wrong.

Router — Policy system selecting tools, recipes, models, or adapters for a task.

Verification — Observed test or validation evidence for a candidate migration.

35. Canonical end-to-end flows

35.1 Fettler

```
Provider source
  ↓
Ingestion
  ↓
ProviderChange
  ↓
Semantic classification
  ↓
Change Graph query/index
  ↓
Impact findings
  ↓
Blast-radius + coverage assessment
  ↓
Migration tasks
  ↓
Router
  ├── deterministic tool/recipe
  ├── specialized adapter/model
  └── frontier escalation
  ↓
Candidate edits
  ↓
Verification
  ↓
Review package
  ↓
Draft PR
  ↓
Reviewer outcome
  ↓
Learning event
  ↓
Graph / recipe / router / model / product improvement
```

35.2 ReGauge

```
Migration objective
↓
Structured MigrationObjective
↓
Repository/system discovery
↓
Change Graph
↓
Constraints + dependencies
↓
Migration graph
↓
Staged plan
↓
Stage approval
↓
Migration tasks
↓
Router
↓
Candidate PR batch
↓
Verification
↓
Review
↓
Stage advance / re-plan / exception
↓
Campaign completion
↓
Learning events
↓
Graph / recipe / router / model / product improvement
```

35.3 Learning

```

Synthetic ground truth
or
Design-partner verified outcome
or
Production verified outcome
↓
LearningEvent
↓
Eligibility + governance
↓
Lesson validation
↓
Destination classification
├─ parser/tool/product fix
├─ graph/retrieval fix
├─ recipe
├─ router policy
├─ calibration
└─ model training
    ↓
    dataset
    ↓
    sealed corpus
    ↓
    external trainer
    ↓
    candidate adapter
    ↓
    hidden holdout evaluation
    ↓
    shadow/canary
    ↓
    human promotion
    ↓
    router eligibility
    ↓
    new outcomes
    ↺
  
```

36. Definition of product maturity

Mendpoint should not describe itself as mature because a demo generates plausible code.

Maturity increases through five levels:

Level 1 — Demonstrable

A narrow migration can be completed end to end under controlled conditions.

Level 2 — Evaluated

The capability has adversarial synthetic coverage, regression tests, and hidden holdouts.

Level 3 — Design-partner ready

Known support boundaries are explicit, review/rollback works, security controls are in place, and the capability can withstand realistic customer variation.

Level 4 — Production reliable

The capability demonstrates durable verification, low severe-regression rates, operational observability, and safe failure behavior across real customer use.

Level 5 — Compounding

Verified outcomes measurably improve routing, recipes, graph intelligence, verification, or specialized models without degrading governance or generalization.

The long-term objective is not simply autonomous migration. It is **trusted, compounding migration intelligence**.

37. Final product statement

Mendpoint is building the migration layer for software.

Fettler turns external provider change into graph-scoped, evidence-backed, verified, reviewable remediation.

ReGauge turns internal modernization intent into dependency-aware, staged, verified migration campaigns.

They share the same Change Graph, router, execution system, verification layer, governance model, evaluation framework, and learning flywheel.

The product advantage compounds when every migration leaves behind more than a merged PR: it leaves behind structured evidence about what changed, what depended on it, what remediation worked, what verification mattered, what model or recipe was sufficient, what reviewers corrected, and how the system should behave the next time.

That is the foundation Mendpoint should build against.